Автоматизация построения признаков для рекомендательного ранжирования

Отчёт по заданию T-Lab 2026, RecSys. Датасет — KION от МТС.

О чём работа

Задание спрашивает, можно ли избавиться от ручного придумывания признаков в рекомендательном ранжировании. Я отвечаю на этот вопрос в три шага.

Во-первых, собран воспроизводимый двухстадийный пайплайн: генераторы кандидатов плюс ранкер на градиентном бустинге с содержательным ручным набором признаков. Во-вторых, признаки описаны **типизированным языком программ (DSL)**, устроенным так, что утечка из будущего в нём невозможна по построению, а не по результатам ревью каждой фичи. Поэтому предлагать признаки можно доверить машине: языковая модель порождает не код, а программу, которая проходит валидацию грамматики до того, как что-либо будет вычислено. В-третьих, поиск по этому языку сравнивается при **строго одинаковом бюджете** между четырьмя стратегиями — случайный поиск, эволюционный поиск, локальная LLM на 7B параметров и большая LLM, — плюс два контрольных подхода: ручной набор, прогнанный через тот же протокол отбора, и систематическая сетка признаков вообще без поиска.

Отдельно проверяется буквальный вопрос из условия — «почему именно семь дней». Вместо жёсткого окна берётся гладкое ядро `exp(-λ·возраст)`, а λ обучается градиентным спуском. Выученный период полураспада можно прямо сравнить с теми окнами, которые человек выбирает на глаз.

Главные выводы собраны в конце отчёта, в разделе «Что сработало и что нет».

Данные и постановка задачи

KION содержит 5 476 251 просмотр за период с 13 марта по 22 августа 2021 года: 962 179 пользователей, 15 963 единицы контента, демография пользователей и метаданные контента (жанры, страны, студии, режиссёры, актёры, ключевые слова, год выпуска, возрастной рейтинг).

Из `watched_pct` сделана вложенная воронка типов событий, похожая на показ → клик → покупку из условия задания:

тип события	условие	доля событий
`start`	любое взаимодействие	100%
`watch`	просмотрено ≥ 30%	~51%
`finish`	просмотрено ≥ 90%	~32%

Важное свойство датасета, которое меняет постановку. В таблице взаимодействий ровно одна строка на пару (пользователь, объект) — колонка называется `last_watch_dt`. Я это проверил: среди пар (пользователь, объект) из целевой недели **ни одна** не встречается в предшествующей истории этого пользователя. То есть повторное потребление в KION невозможно в принципе.

Из этого следуют два вывода. Первый: «прямые связи» user–item, которые условие задания подаёт как ценную группу признаков, здесь вырождены — любой такой признак тождественно равен нулю на кандидатах. Вся персонализация обязана идти через **косвенные связи** (пользователь →

атрибут → объект). Второй: уже просмотренные объекты можно и нужно выкидывать из списка кандидатов, потому что они гарантированно отрицательные. Это поднимает recall кандидатов с 0.23 до 0.35 при том же размере списка.

Разбиение по времени

Задача поставлена как еженедельное предсказание: для даты предсказания `T` история — это всё строго до `T`, целевое окно — `[T, T+7)`, метка равна единице, если пользователь посмотрел объект в целевом окне. Восемь дат идут подряд:

- обучение — 28 июня, 5, 12, 19, 26 июля;
- валидация — 2 августа;
- тест — 9 и 16 августа (две недели, чтобы хватило статистической мощности).

Каждый фолд считает свои признаки по своей истории, поэтому обучающие признаки никогда не видят целевых окон валидации и теста. Признаки валидации видят целевое окно обучения, но для них это прошлое — именно так работает продакшен.

Выборка пользователей

Железо — ноутбук, поэтому пользователи сэмплируются, но **история каждого взятого пользователя сохраняется целиком**, как требует условие. Правило отбора намеренно не смотрит ни на что позже первой даты предсказания: берутся пользователи, у которых к 28 июня 2021 было не меньше пяти событий. Таких 146 747. Результаты измеряются на выборке 60 000 пользователей, поиск признаков ведётся на дешёвой выборке 10 000, а перенос проверяется на всех 146 747.

Статистики айтемов (популярность, конверсии, тренды) считаются по **всей** популяции, а не по выборке: именно такая информация есть у живого сервиса. Статистики пользователя считаются по его собственной истории.

Кандидаты

Кандидаты строятся один раз, на всех 146 747 пользователей, и затем только подмножествуются по пользователям. Благодаря этому все подходы и все масштабы видят буквально одинаковые списки, и сравнение идёт честно. Четыре генератора:

- популярное за 14 дней, топ-150;
- ALS (implicit, 64 фактора, 15 итераций), топ-80 на пользователя;
- item-item ко-просмотры с нормировкой на популярность, топ-80 по восьми последним просмотренным объектам;
- популярное внутри трёх любимых жанров пользователя.

Списки сливаются взвешенной суммой обратных рангов (RRF) и обрезаются до 100 кандидатов на пользователя. Уже просмотренное выбрасывается.

Метрики

Основная метрика — NDCG@10 внутри списка кандидатов: она изолирует качество ранкера, потому что кандидаты у всех подходов одинаковые. Дополнительно приводится **абсолютный** NDCG@10, в котором пользователь наказывается за позитивы, не найденные стадией кандидатов, — это число ближе к тому, что увидит продукт. Также считаются MAP@10, Recall@10, ROC-AUC и LogLoss.

Ранжирующие метрики считаются по группам, в которых есть хотя бы один позитив среди кандидатов: без позитивов NDCG не определён. Ранкер (LambdaRank) на таких группах всё равно получает нулевой градиент, поэтому они исключаются и из обучения. Это стандартная практика для двухстадийных схем, но отмечу её прямо.

Почему в этом пайплайне нет утечек

Это главный инженерный аргумент работы, поэтому он вынесен в отдельный раздел.

Признак в DSL — не код, а JSON-программа. Компилятор программ (`FoldEngine`) получает на входе **только** события строго до даты предсказания, и проверяет это утверждением в конструкторе. Временное окно реализовано как полуинтервал `[T−w, T)` . Отсюда следует, что ни одна программа языка не может заглянуть в будущее — свойство принадлежит движку, а не конкретной фиче. Именно это и даёт право поручить придумывание признаков алгоритму или языковой модели: проверять каждую сгенерированную фичу руками не нужно.

Утверждение подкреплено пятью автоматическими тестами (`tests/test_no_leakage.py`):

1. **Отравление лога будущим.** К событиям добавляются целевая неделя, продублированная с максимальными значениями `watched_pct` и `total_dur` , и синтетические события на 100 дней позже. Все признаки (69 ручных плюс 150 случайных программ) обязаны совпасть побитово с чистым прогоном.
2. **Движок отказывается от будущего.** Передача конструктору лога, который доходит до `T` , должна приводить к ошибке.
3. **Граница окна.** На синтетическом логе с ровно одним событием в день окно `w` обязано посчитать `w−1` событий: возраст 0 — это сама дата предсказания, она относится к целевому окну и в историю не попадает. Проверяется и то, что самое свежее видимое событие — вчерашнее.
4. **Кандидаты — функция прошлого.** Генерация кандидатов на отравленном логе должна дать тот же список. Этот тест нашёл реальную проблему: неустойчивые тай-брейки в сортировках генераторов и ошибку в ко-просмотрах, где «восемь последних объектов» брались по индексу, а не по времени. И то и другое исправлено.
5. **Разрушение метки.** После независимой перестановки меток внутри каждой группы модель на том же наборе признаков обязана работать на уровне случайного порядка. Получается NDCG@10 = 0.0464 против 0.0506 у случайного ранжирования, то есть сигнала не остаётся.

Отдельный тест (`tests/test_engine_equivalence.py`) сравнивает быстрый движок с эталонной реализацией «в лоб» на 250 случайных программах. Быстрый движок восстанавливает атрибутные агрегаты из разложимых компонент айтемного уровня (это на порядок дешевле, чем джойнить миллионы событий), и тест следит, чтобы оптимизация не превратилась в тихую ошибку. Он тоже нашёл реальный баг: суммы длительностей в Float32 теряли разряды.

Исследовательская часть

Язык признаков

Признак записывается как JSON-программа. Атомарная программа — агрегат:

```
{"kind":"agg","anchor":"attr","via":"genres","event":"watch",
 "window":30,"agg":"count","norm":"share_user","pool":"sum"}
```

Читается так: взять события пользователя за последние 30 дней, в которых просмотрено не меньше 30% контента, оставить те, что относятся к объектам, делящим жанр с кандидатом, посчитать их количество, поделить на общее число событий пользователя за те же 30 дней, а если у кандидата несколько жанров — сложить значения по жанрам.

Поля и их домены:

поле	значения	смысл
`anchor`	`user`, `item`, `attr`, `attr_item`	по чему группируем
`via`	9 атрибутов контента	через какой атрибут идёт косвенная связь
`event`	`start`, `watch`, `finish`	уровень воронки
`window`	1, 3, 7, 14, 30, 60, 90, 180, вся история	горизонт
`agg`	12 функций	count, nuniq_item/user/day, mean/max/std_pct, sum/mean_dur, recency, mean_age, trend
`norm`	нет, log1p, per_day, share_user, share_item, z_in_group, rank_in_group	нормировка
`pool`	max, sum, mean, min	свёртка по значениям атрибута у кандидата

Два якоря отвечают за разные вещи. `attr` — это личная косвенная связь: история самого пользователя, отфильтрованная по общему с кандидатом значению атрибута. `attr_item` — поведение толпы внутри этого значения атрибута, то есть «насколько сейчас горяч этот жанр». Отдельно стоит отметить `z_in_group` и `rank_in_group`: они нормируют признак внутри списка кандидатов одного пользователя. Для ранжирования это естественная операция, и, как покажут результаты, именно она оказалась самой полезной.

Композиции: `ratio` считает `a / (b + 1)`, `mul` считает `a · b`.

Валидатор отбрасывает заведомо вырожденные комбинации: `nuniq_user` при группировке по одному пользователю всегда равен единице, `nuniq_item` при группировке по одному объекту — тоже; `per_day` и доли имеют смысл только для счётных агрегатов; `share_user` требует якоря с пользователем, `share_item` — с объектом.

Размер пространства. Валидных атомарных программ — 109 728. С композициями получается около $1.8 \cdot 10^{10}$. Исчерпывающий перебор невозможен даже близко, и это главный аргумент за то, что поиск должен быть умным, а не полным.

Протокол поиска с равным бюджетом

Это ядро экспериментального сравнения. Все стратегии работают в одном и том же цикле; различается только то, как предлагаются программы.

Единица бюджета — одна новая, скомпилированная и проэкранированная программа. Поколение устроено так:

1. стратегия предлагает 32 программы, которых ещё не пробовали;
2. программы компилируются (стоимость замеряется);
3. **экранирование**: обучается одна прокси-модель на «текущий набор + 32 новых», и для каждой новой программы читается её gain по сплитам;
4. берутся 10 лучших по gain, обучается ещё одна прокси-модель, и блок принимается, только если NDCG@10 на валидации вырос.

Поколений 15, то есть бюджет — 480 программ и ровно 30 обучений прокси-модели у каждой стратегии. Прокси-модель одна и та же: LightGBM LambdaRank, lr 0.12, 31 лист, до 300 деревьев с

ранней остановкой. Тестовые фолды внутри поиска не читаются вообще.

Двухступенчатая схема «дешёвое экранирование, затем точная проверка» — это то, как устроены промышленные системы автоматического построения признаков. Она же делает бюджет сопоставимым: стратегия не может выиграть за счёт того, что ей дали больше обучений.

Стратегии:

- **случайный поиск** — равномерная выборка из грамматики. Неинтересный, но обязательный контроль: без него нельзя утверждать, что LLM что-то добавляет.
- **эволюционный поиск** — мутации и скрещивание 12 программ с наибольшим gain, с 15% случайных вливаний для разнообразия.
- **LLM-поиск, Qwen2.5-Coder 7B** — полностью автоматический, модель работает локально на CPU. Нужен, чтобы показать, что цикл замыкается без человека.
- **LLM-поиск, Opus 5** — та же схема, но предложителем выступает большая модель. Все промпты и ответы лежат в репозитории дословно, прогон воспроизводится из них командой без доступа к API.
- **ручной набор через тот же протокол** — контроль на справедливость. Поиск имеет право подстраиваться под валидацию, а ручной набор я писал вслепую. Поэтому 69 ручных программ прогоняются через точно такой же цикл принятия блоков: если преимущество поиска объясняется только доступом к валидации, этот контроль его догонит.
- **систематическая сетка, без поиска** — 480 программ, перечисленных по тем же отношениям, что придумал бы человек, со всеми окнами и несколькими агрегатами, отданных модели целиком. Самая глупая автоматизация, какая может работать. Нужна, чтобы ответить на вопрос «а зачем вообще поиск, если можно посчитать всё и дать бустингу разобраться».

Что видит предложитель на каждом шаге: грамматику языка, описание домена, принятый набор, 18 лучших программ с их gain, программы с нулевым gain («так больше не предлагай») и текущий NDCG@10 на валидации. Больше ничего — в частности, никакой информации с тестовых фолдов.

Обучаемые временные горизонты

Ручной признак фиксирует окно: «покупки за последние 7 дней». Семёрка — догадка. Чтобы снять догадку, жёсткое окно заменяется гладким ядром: вклад события возрастом `d` дней равен `exp(−λ·d)`, а λ — обучаемый параметр.

Всё раскладывается в произведение разреженных матриц. Для отношения `r` суточные счётчики событий живут в небольшой матрице `S_r` (пользователь × день, объект × день, (пользователь, значение атрибута) × день), а разреженное назначение `P_r` сопоставляет строке-кандидату те ключи, на которые она опирается. Признак равен

```
f_r = P_r · (S_r · exp(−λ_r · d))
```

Это дёшево, точно и дифференцируемо по λ. Девять отношений, по два ядра на каждое (быстрое и медленное), поверх — маленький MLP, обучение листовым софтмаксом внутри списка кандидатов. Выученный период полураспада `ln2/λ` сравнивается напрямую с окнами, которые ставит человек.

Дальше выученные λ подставляются обратно: признаки-ядра отдаются тому же бустингу вместо и вместе с ручными окнами. Так проверяется не только то, какой горизонт «хочет» каждое отношение, но и приносит ли это пользу на практике.

Независимо от ядер сделан прямой свип: для каждого из восьми отношений один и тот же счётный признак добавляется к базовой модели со всеми девятью окнами по одному, и записывается

прирост на валидации. Это эмпирический ответ на вопрос «почему именно семь дней» в самой наглядной форме.

Результаты

Фолды и кандидаты

дата предсказания	роль	ранжируемых юзеров	достижимых позитивов	попало в кандидаты	recall кандидатов	групп с позитивом	строк
2021-06-28	train	146 747	145 121	71 414	0.492	39 647	3 964 700
2021-07-05	train	146 747	110 154	41 602	0.378	25 394	2 539 400
2021-07-12	train	146 747	101 222	37 662	0.372	22 793	2 279 300
2021-07-19	train	146 747	91 814	34 359	0.374	20 488	2 048 800
2021-07-26	train	146 747	89 000	33 076	0.372	19 978	1 997 800
2021-08-02	val	146 747	83 036	31 239	0.376	18 524	1 852 400
2021-08-09	test	146 747	80 275	27 534	0.343	16 854	1 685 400
2021-08-16	test	146 747	79 243	28 879	0.364	17 325	1 732 500

Стратегии поиска при равном бюджете

стратегия	сидов	программ	val@128	val@256	val финал	признаков	мин. поиска	ток. вывода LLM
эволюционный поиск	1	480	0.2880	0.2937	0.2937 ± 0.0000	40	8.7	—
LLM-поиск, Opus 5	1	421	0.2907	0.2979	0.2979 ± 0.0000	50	14.0	—
случайный поиск	3	480	0.2893	0.2933	0.2951 ± 0.0015	50	10.9	—

Выученные временные горизонты

отношение	быстрое ядро, дней полураспада	медленное ядро, дней
user_start	5.93	227.14
user_watch	3.65	73.88
item_start	0.71	37.89
item_watch	3.41	13.05
item_finish	1.91	112.54
ugenre	5.12	321.89
udirector	3.85	393.64
ukeyword	3.76	318.68
igenre	12.57	66.57

Ядра внутри градиентного бустинга

набор	признаков	val NDCG@10	test NDCG@10	test абс.
BASE	23	0.2786	0.2354	0.1111
BASE+KERNEL	41	0.3067	0.2423	0.1147
BASE+HUMAN	92	0.2977	0.2342	0.1101
BASE+HUMAN+KERNEL	110	0.3038	0.2364	0.1109

Модель только на ядрах, без бустинга: test NDCG@10 = 0.2232.

Что сработало и что нет

Структура датасета определяет, какие признаки вообще возможны

Условие задания описывает прямые связи user–item как ценную группу признаков. В KION их не существует: таблица взаимодействий уникальна по паре (пользователь, объект), и среди пар целевой недели ни одна не встречается в истории пользователя. Любой признак «сколько раз пользователь смотрел этот объект» тождественно равен нулю на кандидатах. Проверка этого факта сразу дала практический выигрыш: исключение уже просмотренного из списка кандидатов поднимает их recall с 0.227 до 0.351 при том же размере списка.

Вывод, который стоит перенести на другие датасеты: прежде чем строить группы признаков по чек-листу, надо посмотреть, что в данных вообще выразимо.

Самый полезный признак — не величина, а её место в списке кандидатов

Все четыре стратегии поиска независимо сошлись на одном семействе: счётчик недавнего спроса на объект, **нормированный внутри списка кандидатов одного пользователя** (`z_in_group` или `rank_in_group`). У лучших таких программ вклад по сплитам на порядок выше, чем у всего остального.

Это тонкий момент, который легко упустить при ручном проектировании. Модель обучается попарному ранжированию внутри группы, и ей важно не то, что объект популярен вообще, а то, что он популярнее остальных кандидатов этого пользователя. Абсолютное значение популярности

несёт ту же информацию, но в форме, из которой дереву приходится её извлекать сплитами, своими для каждого пользовательского профиля.

В ручном наборе такая нормировка была — но применённая к счётчику показов за семь дней. Поиск нашёл, что лучше работает число **уникальных пользователей** за **три дня**. То есть человек угадал форму признака, но не угадал ни агрегат, ни окно, а разница между этими вариантами в разы больше, чем между наличием и отсутствием самой формы.

Признаки одного пользователя бесполезны в ранжировании сами по себе

В принятых наборах почти нет программ с якорем `user`. Причина структурная: в listwise-ранжировании внутри группы такой признак — константа, и упорядочить кандидатов он не может. Он работает только через взаимодействия с айтемными признаками, и именно в такой форме поиск его и нашёл: произведение активности пользователя на относительную популярность объекта попало в пятёрку лучших программ по вкладу.

Это объясняет, почему добрая половина типового ручного набора — агрегаты активности пользователя — в двухстадийной схеме с ранжированием внутри группы почти не окупается.

Явные произведения признаков в основном избыточны

Блок с восемью произведениями «пользовательская величина × относительная популярность» получил высокие оценки на экранировании, но был **отклонён**: на валидации качество не выросло. Объяснение простое — ансамбль деревьев строит произведения сам, и подавать их готовыми имеет смысл только тогда, когда нужное произведение требует слишком многих сплитов. Полезными оказались не произведения, а отношения: конверсии между уровнями воронки и отношения одной и той же величины на двух горизонтах («доля месячного охвата, набранная за три дня»).

Практический вывод: при автоматической генерации признаков композиции типа `mul` почти не нужны, а `ratio` нужны.

Поиск нашёл границы собственного языка

К середине бюджета плечо с большой моделью перестало улучшать валидацию: пять блоков подряд отклонены. Разбор того, чего не хватало, дал два семейства, которые в плоском языке **невыразимы в принципе**:

1. **Агрегат от агрегата.** «Насколько этот объект популярнее того, что этот пользователь обычно смотрит» требует сначала измерить каждый объект истории айтемным признаком, потом свернуть это в профиль пользователя, потом сравнить кандидата с профилем. Плоская грамматика умеет только один уровень.
2. **Сегмент пользователя.** «Популярно среди женщин 25–34» требует якоря по атрибуту пользователя, которого в языке не было вовсе: атрибуты были только у контента.

Оба семейства добавлены как расширение языка и измерены отдельно, уже вне сравнения стратегий, чтобы не менять условия задним числом. Результаты — в таблице расширения языка.

Это, пожалуй, самый интересный методологический результат работы. Автоматический поиск полезен не только тем, что находит хорошие признаки, но и тем, что его насыщение — измеримый сигнал о том, что пора расширять язык, а не бюджет.

«Почему именно семь дней»

Это вопрос из условия задания, и он получил три независимых ответа, которые совпали.

Прямой свип. Для каждого отношения один и тот же счётный признак добавлялся к базовой модели со всеми девятью окнами по одному. Оптимумы оказались далеко от привычных семи и тридцати дней и разными для разных отношений. Таблица и рисунок — в разделе результатов.

Дискретный поиск. Плечо с большой моделью, которому было прямо сказано варьировать окна осознанно, сошлось на распределении горизонтов, где айтемные программы сидят на трёх и семи днях, а вкусовые — на всей истории. Окно в один день получило ровно нулевой вклад у всех агрегатов: данных в сутках слишком мало. Случайный поиск, напротив, разбросал принятые программы по окнам 60–180 дней, и качество у него вышло ниже.

Градиентный спуск. Обучаемые ядра `exp(-λ·возраст)` дали такую же картину, но уже в виде числа, а не выбора из списка:

отношение	быстрое ядро	медленное ядро
популярность объекта	меньше суток	около месяца
глубокие просмотры объекта	3 дня	2 недели
активность пользователя	6 дней	7 месяцев
вкус к жанру	5 дней	около года
вкус к режиссёру	4 дня	больше года
вкус к теме	4 дня	около года

(точные значения — в таблице выученных горизонтов)

Картина устойчивая и содержательно понятная: **спрос на контент — быстро забывающая величина, а вкус человека — почти не забывающая.** Ручной набор с окнами 7 и 30 дней промахивается в обе стороны одновременно: для популярности семь дней — это слишком долго и сигнал размывается, а для жанровых предпочтений тридцать дней — это слишком коротко и половина профиля выбрасывается.

Важно, что выученные горизонты не остаются отдельной моделью. Они возвращаются в язык как обычная программа с мягким окном (`decay`), то есть компилируются, кэшируются и проходят те же тесты на утечки, что и любой другой признак, а значит их можно просто добавить к бустингу и измерить на любом масштабе.

Чего стоит LLM в цикле

Плечо с большой моделью потратило 15 вызовов и вышло на свой максимум примерно на 190 предложениях из доступных 480 — быстрее всех остальных. Дальше оно насытилось: пять блоков подряд были отклонены, и итог не изменился до конца бюджета. Интересно, что из-за дублей с уже опробованными программами это плечо израсходовало **меньше** бюджета, чем остальные (421 программа против 480), и всё равно осталось впереди по валидации.

Локальная модель на 7 миллиардов параметров на том же промпте повела себя иначе и показала, что сила предложителя имеет значение:

- примерно половина её предложений нарушала грамматику — чаще всего она ставила `via` и `pool` при якорях `user` и `item`, где их быть не должно, и писала строку `"None"` там, где JSON требует `null`;
- потребовался раунд починки, в котором модели дословно показывали вердикт валидатора и просили исправиться (этот раунд доступен обоим бэкендам одинаково; большой модели он ни разу не понадобился);
- на процессоре она выдаёт около 9 токенов в секунду, то есть примерно пять минут на одно поколение, что сопоставимо со временем самого поиска.

Отсюда практический вывод для такой схемы: типизированный язык с валидатором — это не бюрократия, а то, что делает слабую модель вообще пригодной к использованию. Без грамматики её предложения пришлось бы проверять руками, и вся экономия пропала бы.

Ограничения

Кандидаты ограничивают потолок. Recall стадии кандидатов — 0.33–0.37 на тестовых неделях. Всё, что выше по тексту, измеряет качество ранкера внутри этого потолка; абсолютные NDCG приведены отдельно и, разумеется, втрое ниже. Улучшение генераторов дало бы больше, чем любые из рассмотренных признаков, но это другая задача.

Один датасет. Все выводы получены на KION. Структурные особенности KION (нет повторных просмотров, один регион, 16 тысяч объектов, пять месяцев истории) могли повлиять на то, какие признаки оказались полезными. В частности, короткий горизонт айтемных признаков может быть следствием того, что каталог активно пополнялся в этот период.

Поиск идёт на выборке 10 000 пользователей. Это позволяет уложить прогон стратегии в десять минут, но найденный набор может быть переобучен под маленькую выборку. Проверка переноса на все 146 747 пользователей включена в отчёт именно для этого.

Прокси-оценщик не равен итоговой модели. Внутри поиска обучается LightGBM с более высоким learning rate и меньшим числом листьев. Корреляция между прокси и итоговой моделью не измерялась отдельно; переносимость видна только по итоговой таблице.

Композиции в DSL не нормируются. `ratio` и `mul` не имеют поля `norm`, поэтому, например, «отношение двух величин, затем ранг внутри списка» записать нельзя — только «отношение величин, каждая из которых уже нормирована».

У LLM-плеча меньше сидов. Прогон с большой моделью сделан один раз, с фиксированным набором транскриптов. Разброс по сидам для него не измерен, в отличие от случайного и эволюционного поиска.

Экранирование по gain — эвристика. Отбор десяти программ из блока идёт по split gain, то есть по вкладу в обучающие сплиты, а не по приросту на валидации. Это дёшево и одинаково для всех стратегий, но предвзято в пользу признаков с большим числом уникальных значений.

Воспроизведение

```
python -m venv .venv && . .venv/bin/activate     # python 3.12
pip install -r requirements.txt
./run_all.sh
```

`run_all.sh` скачивает KION, собирает признаковые таблицы и кандидаты, прогоняет тесты на утечки, затем все стратегии поиска, свип окон, обучаемые ядра, итоговые сравнения и строит таблицы с рисунками. Полный прогон на ноутбуке с восьмиядерным процессором занимает около шести часов; самые долгие шаги — итоговые обучения на выборке 60 000 пользователей.

Плечо с большой языковой моделью воспроизводится без доступа к API: все промпты и ответы лежат в `runs/llm_opus5_none_s0/` дословно, и прогон повторяет их из файлов. Плечо с локальной моделью требует запущенного `ollama` с `qwen2.5-coder:7b-instruct`; если сервер недоступен, шаг пропускается.

Отдельные шаги:

```
python scripts/prepare.py                # CSV -> parquet, таблицы атрибутов
python scripts/build_candidates.py       # кандидаты и метки для восьми фолдов
python tests/test_no_leakage.py          # пять тестов на утечки
python tests/test_engine_equivalence.py  # быстрый движок против эталонного
python scripts/run_search.py --strategy evolution --seed 0
python scripts/run_window_sweep.py       # рисунок с окнами
python scripts/run_kernels.py            # обучаемые горизонты
python scripts/run_baselines.py --scale main
python scripts/run_final.py --scale main  # единственный скрипт, читающий тест
python scripts/run_nested_extension.py    # расширение языка
python scripts/make_tables.py && python scripts/make_figures.py
```

Структура репозитория

```
src/tlab/
  config.py      даты фолдов, масштабы выборок, пути
  dataset.py     загрузка, выборка пользователей, генераторы кандидатов
  dsl.py         язык признаков: грамматика, валидатор, компилятор, эталонный движок
  space.py       перечисление, сэмплирование и мутации программ
  human.py       ручной набор, систематическая сетка, расширения языка
  pipeline.py    сборка матриц: базовые признаки, кэш программ, фолды
  rank.py        LightGBM LambdaRank, обучение и оценка
  metrics.py     NDCG/MAP/Recall в двух видах, парный бутстрэп
  search.py      протокол поиска с равным бюджетом и стратегии без LLM
  llm.py         грамматика в промпте, парсер ответов, два бэкенда
  kernels.py     разреженная факторизация и обучаемые ядра затухания
scripts/         запуск каждого этапа
tests/           тесты на утечки и на эквивалентность движков
runs/            результаты прогонов в JSON и транскрипты LLM
reports/         таблицы и рисунки
```